Precision Support for Row-Major × Row-Major GPU Matrix Multiplication: FP16, BF16, and FP32 on Tensor Cores

ganesh@cs.utah.edu
(written with Claude Code)

September 25, 2026

Abstract

This report studies matrix multiplication $C = A \times B$ where both A and B arrive in **row-major** layout, restricted to three precisions: FP16, BF16, and FP32. Integer (INT8/IMMA) matrix multiplication is out of scope and has been commented out of `tc_matmult/leimao-mm.cu` accordingly. Every claim about what the hardware and CUDA actually support here is verified directly against the CUDA 12.4 `mma.hpp` header shipped on this machine (path in Section 2), not general recollection — exact fragment types, tile shapes, and accumulator types are quoted from it. FP16 is fully exercised, working code (`leimao-mm.cu`); BF16 and the FP32/TF32 path are supported by the same header but not exercised by that file. Section 5 answers directly: no SIMT-core (ordinary per-thread CUDA) matrix multiplication happens anywhere in this file — every GPU-side multiply-accumulate goes through tensor cores.

Contents

1 Scope and What "Row-Major, Row-Major" Means Here **2**

2 FP16 (HMMA): Working, and Already in the File **2**
2.1 What the header actually defines . . . 2
2.2 How the multiplication is realized . . . 3

3 BF16: Supported by the Header, Not Exercised by This File **3**

4 FP32: Not on Tensor Cores at Full Precision — Only via TF32 **4**
4.1 What the header does *not* define . . . 4
4.2 What the header does define: TF32 . . . 4
4.3 If true full-precision FP32 GEMM is actually required . . . 4

5 Is Any of This SIMT-Core Matrix Multiplication? **5**

6 SIMT vs. Tensor Core: A Real Benchmark (FP16) **5**

7 FP32 (SIMT) vs. BF16 (Tensor Core): A Second Real Benchmark **6**

8 Summary Table **6**

9 Observing the FP16 Tensor-Core Kernel Under `nixnan` **7**
9.1 The Reduced-Size (128^3) Run . . . 7

9.2 Instrumentation Overhead 8

10 Forcing FP16 Overflow: Tensor Core vs. SIMT **9**
10.1 Two new files 9
10.2 Results: host-side sanity check (independent of nixnan) 9
10.3 Results: caught by nixnan 10

11 Conclusion **10**

1 Scope and What "Row-Major, Row-Major" Means Here

`leimao-mm.cu` (Lei Mao's WMMA GEMM example, copied verbatim into this repository) exposes its GEMM through a column-major-plus-transpose-flag interface:

```
launch_wmma_mm(A, B, C, m, n, k, is_A_transpose, is_B_transpose, stream);
```

Its own header comment states the convention precisely: *"All the data in the matrices are stored in a column-major order... `WMMA_FRAG_LAYOUT_A: row_major` if A is transposed, otherwise `col_major`."* This is a standard trick, not a real transpose: an $M \times K$ matrix stored **row-major** occupies *exactly* the same bytes in memory as its $K \times M$ transpose stored **column-major**. So "`is_A_transpose=true`" in this code does not mean "compute A^T" in any mathematical sense that changes the answer — it means "the data `A` points to is laid out row-major; load it that way." The code confirms this directly: setting the flag selects `nvcuda::wmma::row_major` as the fragment load layout, and adjusts the leading dimension accordingly (`lda = is_A_transpose ? k : m`, correct for a row-major $M \times K$ array).

Consequence for this study: row-major A and row-major B is obtained with

```
launch_wmma_mm(A, B, C, m, n, k, /*is_A_transpose=*/true, /*is_B_transpose=*/true,
    stream);
```

— the branch the demo's own `print_test_header` labels "`C = A^T * B^T`". That label is accurate to the code's internal naming convention but *not* a mathematical transpose of the matrices this report cares about: it is exactly $C = A \times B$ with both operands read as ordinary row-major arrays.

2 FP16 (HMMA): Working, and Already in the File

2.1 What the header actually defines

From `crt/mma.hpp`, CUDA 12.4 (full path on this machine: `/usr/local/cuda-12.4/targets/x86_64-linux/include/crt/mma.hpp`): `wmma::fragment<matrix_a/matrix_b, 16, 16, 16, __half, row_major|col_major>` is defined, with `load_matrix_sync` for both layouts. Two families of `mma_sync` overloads exist for the accumulator: one into `fragment<accumulator, 16, 16, 16, __half>` (FP16 accumulation) and one into `fragment<accumulator, 16, 16, 16, float>` (FP32 accumulation) — *both* are real, hardware-backed instruction sequences, not a software emulation of one from the other. The same three families of tile shapes appear for FP16 as for every other supported type here: $16 \times 16 \times 16$, $32 \times 8 \times 16$, and $8 \times 32 \times 16$.

`leimao-mm.cu` always instantiates `launch_wmma_mm<__half, float>` — i.e. it always chooses the

FP32-accumulator HMMA path, at the $16 \times 16 \times 16$ tile shape, and never exercises the FP16-accumulator variant even though the hardware/header support it.

2.2 How the multiplication is realized

One CUDA *warp* (32 threads acting as one unit) computes one 16×16 output tile per iteration of the outer loop, striding over K in steps of 16:

```
warpM, warpN <- this warp's 2D tile coordinates (from blockIdx/threadIdx)
acc_frag: fragment<accumulator, 16,16,16, float> <- fill_fragment(0)
for ki = 0, 16, 32, ..., k-16:
    a_frag: fragment<matrix_a, 16,16,16, half, row_major>
    b_frag: fragment<matrix_b, 16,16,16, half, row_major>
    load_matrix_sync(a_frag, A + offset(warpM, ki), lda) # row-major load
    load_matrix_sync(b_frag, B + offset(ki, warpN), ldb) # row-major load
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one HMMA tensor-core op
c_frag: fragment<accumulator, 16,16,16, float> <- load existing C tile
for each element e in c_frag: c_frag[e] = alpha*acc_frag[e] + beta*c_frag[e]
store_matrix_sync(C + offset(warpM, warpN), c_frag, ldc, mem_col_major)
```

Every multiply-accumulate over the $16 \times 16 \times 16$ tile happens in a single `mma_sync` call — one tensor-core HMMA instruction, not 4096 separate multiply-adds issued by ordinary CUDA cores. `load_matrix_sync` and `store_matrix_sync` handle the row-major↔fragment layout translation entirely inside the intrinsic; the kernel code never manually transposes anything.

3 BF16: Supported by the Header, Not Exercised by This File

Yes, BF16 is supported — on Ampere (compute capability 8.0+, which includes `sm_86`, the target this file already builds for) and confirmed directly in the same header:

```
wmma::fragment<matrix_a/matrix_b, 16, 16, 16, __nv_bfloat16, row_major|col_major>
```

is defined, at all three tile shapes (`16×16×16`, `32×8×16`, `8×32×16`), with full `load_matrix_sync` support for both layouts.

One structural difference from FP16, confirmed by the header: only `mma_sync` overloads accumulating into `fragment<accumulator, 16,16,16, float>` exist for BF16 — there is *no* BF16-accumulator overload. BF16 tensor-core matrix multiplication on this hardware **always** accumulates in FP32; there is no choice to make, unlike FP16.

`leimao-mm.cu` never instantiates a BF16 path — `__nv_bfloat16` does not appear anywhere in the file. It now exists as a separate file, `leimao-tc-variant-bf16.cu`: a direct, mechanical port of Section 2's kernel with `__half` swapped for `__nv_bfloat16` (accumulator forced to `float`, the only type available), restricted like the SIMT files to row-major × row-major (Section 1):

```
acc_frag: fragment<accumulator, 16,16,16, float> <- fill_fragment(0) # float is
    mandatory
for ki = 0, 16, 32, ..., k-16:
    a_frag: fragment<matrix_a, 16,16,16, __nv_bfloat16, row_major>
    b_frag: fragment<matrix_b, 16,16,16, __nv_bfloat16, row_major>
    load_matrix_sync(a_frag, A + offset(warpM, ki), lda)
    load_matrix_sync(b_frag, B + offset(ki, warpN), ldb)
```

```
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one BF16 tensor-core op
# store identical to FP16 case, c_frag is float throughout
```

Verified working: correctness-checked against a CPU reference (no diff-ratio warning, same 0.01 tolerance as every other file here) and benchmarked in Section 7.

4 FP32: Not on Tensor Cores at Full Precision — Only via TF32

This is the precision where the answer is not "yes, same pattern, different type."

4.1 What the header does *not* define

Searching `crt/mma.hpp` for a tensor-core fragment holding true, full-mantissa `float` operands (23-bit mantissa, IEEE-754 single precision) for `matrix_a`/`matrix_b` finds nothing. **Tensor cores on this GPU cannot multiply two full-precision FP32 matrices directly.** There is no WMMA instruction for it, at any tile shape.

4.2 What the header does define: TF32

`wmma::fragment<matrix_a/matrix_b, 16, 16, 8, precision::tf32, row_major|col_major>` — note the K dimension is **8**, not 16, unlike every other type in this report. `load_matrix_sync` for this fragment still takes a plain `const float*` pointer: the caller's data in host/device memory stays ordinary FP32 the whole time. The precision reduction — TF32 keeps FP32's 8-bit exponent but truncates the mantissa to 10 bits, matching FP16's mantissa width while keeping FP32's exponent range — happens *inside* the load intrinsic itself, not as a separate conversion step the caller writes. The accumulator, `fragment<accumulator, 16,16,8, float>`, is full-precision `float`, and `mma_sync` overloads for all four row/col-major combinations of `matrix_a`/`matrix_b` are present, confirming TF32 GEMM is fully supported for row-major operands, exactly like Sections 2 and 3.

`leimao-mm.cu` does not exercise this path either — no `precision::tf32` anywhere in the file. A TF32 version of the kernel:

```
acc_frag: fragment<accumulator, 16,16,8, float> <- fill_fragment(0)
for ki = 0, 8, 16, ..., k-8: # K-tile is 8, not 16
    a_frag: fragment<matrix_a, 16,16,8, precision::tf32, row_major>
    b_frag: fragment<matrix_b, 16,16,8, precision::tf32, row_major>
    load_matrix_sync(a_frag, A_float_ptr + offset(warpM, ki), lda) # FP32 in, TF32
        internally
    load_matrix_sync(b_frag, B_float_ptr + offset(ki, warpN), ldb)
    mma_sync(acc_frag, a_frag, b_frag, acc_frag) # one TF32 tensor-core op
# store: c_frag is float, full precision, no further loss on the output/accumulate side
```

4.3 If true full-precision FP32 GEMM is actually required

Then it cannot run on tensor cores on this hardware at all — it has to be an ordinary SIMT kernel using plain `FFMA` instructions on the regular CUDA cores, ordinary per-thread arithmetic, no `wmma` namespace involved:

```
# one thread per output element (a real kernel would tile through shared memory;
# this is the conceptual/unoptimized form)
row, col <- this thread's output coordinates
```

```
acc: float = 0
for ki = 0, 1, 2, ..., k-1:
    acc += A[row, ki] * B[ki, col] # ordinary FFMA, full 23-bit mantissa throughout
C[row, col] = acc
```

`leimao-mm.cu` does not contain this kernel either (see Section 5) — its only full-precision FP32 code is the CPU-side reference function, not a GPU kernel at all.

5 Is Any of This SIMT-Core Matrix Multiplication?

No. `leimao-mm.cu` contains exactly one `__global__` kernel, `wmma_gemm_a_col_major_b_col_major`, and every multiply-accumulate inside it happens via `nvcuda::wmma::mma_sync` — a tensor-core instruction. There is no kernel anywhere in this file that computes a matrix product using ordinary per-thread CUDA arithmetic (`FFMA`, `HFMA2`, or otherwise) on the GPU.

The one non-tensor-core matrix multiplication in the file, `mm_a_col_major_b_col_major`, is **not a GPU kernel at all**: it has no `__global__` or `__device__` qualifier, takes no thread indices, and is called directly from `main()` on the host. It is a plain, serial, triple-nested-loop C++ function that runs on the **CPU**, used purely to compute a correctness reference (`get_avg_abs_diff_ratio` compares it against the tensor-core result) — its runtime is never measured or reported, unlike the tensor-core kernel's (`measure_performance`, cudaEvent-timed).

Summary: 100% of GPU-side matrix multiplication in `leimao-mm.cu` is tensor-core work. 0% is SIMT-core. A true SIMT GEMM kernel, as sketched in Section 4, does not exist in `leimao-mm.cu` in any form — it has now been written from scratch as a separate pair of files, `leimao-simt-variant-fp32.cu` and `leimao-simt-variant-fp16.cu`, benchmarked directly against the tensor-core path in Section 6.

6 SIMT vs. Tensor Core: A Real Benchmark (FP16)

`leimao-simt-variant-fp32.cu` and `leimao-simt-variant-fp16.cu` are the SIMT kernels Section 5 found missing from `leimao-mm.cu` — a standard shared-memory tiled GEMM (32×32 tiles, one thread per output element, ordinary `FFMA` throughout, ordinary CUDA (SIMT) cores, no `wmma` namespace anywhere), restricted to exactly the row-major $\times$ row-major case this report studies (Section 1). The FP16 variant matches `leimao-mm.cu`'s own accumulation choice for a fair comparison: `__half` inputs, `float` accumulation (via `__half2float` inside the inner loop), `float` output — the same precision contract as `launch_wmma_mm<__half, float>`, so the only thing actually being compared is *which hardware path performs the multiply-accumulate*, not a difference in numerical precision along the way. Both SIMT files reuse the exact same `measure_performance` (cudaEvent-timed, 10 warmups, 10 measured repeats) and $1024 \times 1024 \times 1024$ problem size as `leimao-mm.cu`, and both were checked for correctness against a CPU reference before timing (no "high average absolute diff ratio" warning printed by either — i.e. within the same 0.01 average relative-error tolerance `leimao-mm.cu` itself uses).

	Latency (ms)	
Tensor core (HMMA, `leimao-mm.cu`, "C = A^T * B^T")	0.177–0.178	(3 runs)
SIMT (`leimao-simt-variant-fp16.cu`)	0.864–0.866	(3 runs)

Averaging each: tensor core ≈ 0.178 ms, SIMT ≈ 0.865 ms — **the tensor-core path is ≈4.9× faster** than this tiled SIMT kernel, same GPU, same problem size, same input/accumulation precision, same 1024^3-element problem. This is a real, measured number from this machine, not a textbook or vendor figure; it should not be read as "the" tensor-core speedup in general — a more aggressively tuned SIMT kernel (larger tiles, register blocking, double-buffered shared memory, vectorized loads) would close some of this gap, and the gap itself grows or shrinks with problem size and GPU generation. What it does establish concretely: even a reasonably standard, textbook-tiled SIMT GEMM is not close to tensor-core throughput on this hardware for this problem size — the $16 \times 16 \times 16$-element HMMA instruction Section 2 walks through is doing, per issue, what this SIMT kernel needs $16^3 = 4096$ separate scalar `FFMA`-equivalent operations to accomplish.

7 FP32 (SIMT) vs. BF16 (Tensor Core): A Second Real Benchmark

A second, deliberately *cross-precision* comparison: `leimao-simt-variant-fp32.cu` (Section 6's FP32 SIMT kernel, full-precision, ordinary CUDA cores) against `leimao-tc-variant-bf16.cu` (Section 3's BF16 kernel, tensor cores). Unlike Section 6, this pairing is not precision-matched on purpose — it is the practical question a would-be user of this GPU actually faces: *if my data is FP32, is it worth converting to BF16 to use the tensor cores, or should I just run FP32 on the SIMT cores as-is?* Same 1024^3 problem, same `measure_performance` methodology (10 warmups, 10 measured repeats), 3 runs each, both checked for correctness against a CPU reference before timing (no diff-ratio warning from either):

	Latency (ms)	
FP32 SIMT (`leimao-simt-variant-fp32.cu`)	0.844–0.846	(3 runs)
BF16 tensor core (`leimao-tc-variant-bf16.cu`)	0.177	(3 runs, identical each time)

Averaging: FP32 SIMT ≈ 0.845 ms, BF16 tensor core ≈ 0.177 ms — ≈4.8× **faster**, essentially the same margin as Section 6's precision-matched FP16 comparison (≈ 4.9×), and the BF16 number itself is within measurement noise of the FP16 tensor-core number there (both ≈ 0.177–0.178 ms) — consistent with Section 3's header-level finding that FP16 and BF16 share the same tile shape and accumulator type on this hardware, so there is no reason to expect their raw tensor-core throughput to differ, and it doesn't. The practical answer to the question this section opened with: on this GPU, at this problem size, converting FP32 data to BF16 and using the tensor cores is close to 5× faster than computing directly in FP32 on the SIMT cores — at the cost of BF16's reduced mantissa (8 bits, versus FP32's 23), the same accuracy trade Section 4 describes for TF32.

8 Summary Table

Precision	Tensor core?	Fragment type	Accumulator	Tile ($M \times N \times K$)
FP16	Yes (HMMA)	`__half`	`__half` *or* `float`	16×16×16 (+2 more)
BF16	Yes	`__nv_bfloat16`	`float` only	16×16×16 (+2 more)
FP32 (TF32)	Yes	`precision::tf32`	`float`	16×16×**8**
FP32 (full)	No	— (SIMT only)	`float`	N/A

Precision	Exercised in `tc_matmult/`?
FP16, tensor core	Yes — `leimao-mm.cu`'s only working GEMM path (FP32 accumulator)
FP16, SIMT	Yes — `leimao-simt-variant-fp16.cu` (Section 6)
FP32, SIMT	Yes — `leimao-simt-variant-fp32.cu`
BF16, tensor core	Yes — `leimao-tc-variant-bf16.cu` (Section 7)
FP32 (TF32)	No — supported by the header/hardware, absent from this directory

A note beyond the three precisions in scope. The same header also defines `fragment<matrix_a/matrix_b, 8, 8, 4, double, row_major|col_major>` — double-precision (DMMA) tensor-core matrix multiplication exists too, at a much smaller 8×8×4 tile. Out of scope for this report, but worth knowing it isn't only FP16/BF16/TF32/INT8 that get tensor-core paths on this architecture family.

9 Observing the FP16 Tensor-Core Kernel Under `nixnan`

As a further exercise, the FP16 tensor-core kernel (`leimao-mm.cu`) was run under `nixnan`, the project's NVBit-based FP exception/histogram instrumentation tool (see `rd_nixnan/` elsewhere in this repository), to (a) confirm it can observe tensor-core instructions at all, and (b) measure what that observation actually costs.

9.1 The Reduced-Size (128^3) Run

Why a reduced size was needed. The first attempt instrumented the original 1024^3 kernel with `BIN_SPEC_FILE`'s finest granularity, `"count": 1` (report every single occurrence of a value in a watched exponent range). That kernel executes on the order of 262,000 HMMA instructions per launch; at `count:1` this produced an overwhelming volume of report messages through `nixnan`'s channel, and the runs were killed after running for over 10 minutes with no output.

The four 128^3 files. Every kernel used elsewhere in this report was copied to a $128 \times 128 \times 128$ variant, with only `matrix_size_m/n/k` changed from `1024` to `128` (everything else — kernel code, tiling, the warmup/timed-repeat harness — identical to its 1024^3 original):

1024^3 original	128^3 variant
`leimao-mm.cu` (FP16, tensor core)	`leimao-mm-128x128.cu`
`leimao-simt-variant-fp32.cu`	`leimao-simt-variant-fp32-128x128.cu`
`leimao-simt-variant-fp16.cu`	`leimao-simt-variant-fp16-128x128.cu`
`leimao-tc-variant-bf16.cu`	`leimao-tc-variant-bf16-128x128.cu`

This is 1/512 the work of the 1024^3 kernels ($(128/1024)^3$), which made `count:1` histogram observation tractable: the longest `nixnan` run in Section 9.2 below completes in well under a minute, versus the 1024^3 attempt's 10+-minute hang.

Correctness. All four 128^3 binaries were rebuilt and rerun standalone (no `nixnan`), using each file's existing `assert`-based check against its CPU reference (`leimao-mm-128x128.cu`'s `array_equal` comparison for the HMMA path, the SIMT/BF16 files' own `cpu_reference` comparison). All four ran to completion without tripping their assertion, i.e. the size reduction did not change correctness for any of the four kernel variants.

Baseline (no `nixnan`) performance at 128^3. For reference, all four kernels' own reported latency at this size, row-major × row-major, no instrumentation:

Kernel	128^3 Latency	1024^3 Latency (for comparison)
FP16, tensor core (`leimao-mm-128x128`)	0.008 ms	≈ 0.178 ms
BF16, tensor core (`leimao-tc-variant-bf16-128x128`)	0.008 ms	≈ 0.177 ms
FP32, SIMT (`leimao-simt-variant-fp32-128x128`)	0.011 ms	≈ 0.845 ms
FP16, SIMT (`leimao-simt-variant-fp16-128x128`)	0.011 ms	≈ 0.865 ms

At 1024^3 (Sections 6 and 7) tensor cores beat SIMT by ≈ 4.8–4.9×. At 128^3 that gap collapses to ≈ 1.4× (0.011/0.008): at this size the problem is small enough that fixed per-launch overhead (kernel launch, grid setup) dominates for every kernel, tensor-core and SIMT alike, and there isn't enough arithmetic work left for the tensor cores' raw throughput advantage to show through. The 128^3 size is used here purely to make `nixnan`'s `count:1` observation tractable, not as a size at which the tensor-core-vs-SIMT comparison itself is meaningful — that comparison is what Sections 6 and 7 are for.

9.2 Instrumentation Overhead

Confirmation: `nixnan` does instrument tensor-core instructions. With `TOOL_VERBOSE=1 HISTOGRAM=1`, `nixnan`'s instruction-matching table was confirmed to catch `HMMA.16816.F32` directly (e.g. `"Histogram instrumenting:  HMMA.16816.F32 R8, R12.reuse, R22, R8 ;"`) — tensor-core MMA instructions are not invisible to this instrumentation, despite operating on packed fragment registers rather than the scalar FP values `nixnan` was originally built to watch.

Four instrumentation levels, one kernel. All runs below are the row-major A × row-major B case ("`C = A * B`" in `leimao-mm.cu`'s output, the one this report is scoped to), on the 128^3 kernel, using the same 10 warmup / 10 timed-repeat `measure_performance` harness as every other benchmark in this report. Full commands, environment variables, and raw logs are in `tc_matmult/Logs/`.

Instrumentation level	HMMA Latency	Slowdown
No `nixnan` (baseline)	0.008 ms	1×
`SAMPLING=128` + histogram (`count:1,doublings:6`)	0.015 ms	≈1.9×
Unsampled, default exceptions only	0.706 ms	≈88.3×
Unsampled, exceptions + histogram (`count:1,doublings:6`)	257.009 ms	≈32,126×

Why the `SAMPLING=128` row is misleadingly cheap. `SAMPLING` gates instrumentation by `kernel_count % SAMPLING == 0`. Unlike `rd_nixnan`'s workloads (1600+ kernel launches), `leimao-mm-128x128` launches its kernel only ∼88 times total (4 transpose combinations × (1 correctness check + 10 warmups + 10 timed repeats)). With `SAMPLING=128`, only launch #0 — the very first correctness check, never a *timed* launch — is ever instrumented; every one of the 80 timed repeats that `measure_performance` actually reports on runs completely uninstrumented. The 0.015 ms figure is therefore close to true baseline by construction, not evidence that histogramming is cheap. The remaining three rows, run *unsampled* (every launch instrumented, matching what a real observation session would use), show the real cost.

The exponent ranges watched. The histogram runs used `count:1, doublings:6` (report on the 1st, 2nd, 4th, 8th, …, 64th occurrence in a bin, then reset), watching FP16 exponent

ranges $[-9, -8]$ and $[7, 8]$. These were chosen empirically, not by convention: a plain `HISTOGRAM=1` run (no `BIN_SPEC_FILE`) against this exact 128^3 kernel reports `"Exponent range for f16: [-9, 8]"` — narrower at the low end than the 1024^3 kernel's `[zero, 8]`, since fewer samples means less chance of hitting the rarest, near-zero bin. The project's other usual convention of watching FP16's full-range extremes, $[-14, -13]$ and $[14, 15]$, would see zero occurrences on this workload (`uniform(-256,256)`-sourced input data never reaches those exponents here) and was not used.

Log volume. The unsampled default-exceptions-only run produced a 6,798-line log, committed in full. The unsampled exceptions+histogram run produced 1,054,204 lines (336 MB) — far too large to commit; `Logs/` holds a truncated summary (header, first/last lines, and full per-threshold-level counts computed over the complete original file) instead. The `SAMPLING=128` run's 99,310-line (32 MB) log is likewise truncated the same way. Both are captured in `tc_matmult/Logs/`, named for their configuration.

10 Forcing FP16 Overflow: Tensor Core vs. SIMT

Every run so far (Section 9) produced *zero* FP exceptions of any kind. That is a property of the input magnitude, not of the hardware: `leimao-mm.cu`'s HMMA path accumulates in `float` (`launch_wmma_mm<__half, float>`), and the SIMT comparison kernels (Sections 6, 7) were deliberately built to match that same wide-accumulator contract for a fair timing comparison. This section asks the question directly: raise the input magnitude and force a genuinely *narrow* (`__half`) accumulator instead — does a tensor core overflow then, and does an ordinary SIMT-core FP16 kernel overflow on its own, with zero tensor-core instructions involved at all?

10.1 Two new files

- `leimao-mm-128x128-overflow.cu`: the same WMMA kernel as `leimao-mm-128x128.cu`, but every transpose combination now runs *both* `launch_wmma_mm<__half, float>` (control, wide accumulator) *and* `launch_wmma_mm<__half, __half>` (narrow accumulator) back to back, with the output matrix scanned for NaN/Inf independent of nixnan as a sanity check.
- `leimao-simt-variant-fp16-128x128-overflow.cu`: unlike `leimao-simt-variant-fp16-128x128.cu` (Section 6, which accumulates in `float` via `__half2float`), this kernel accumulates in `__half` the entire way — ordinary `operator*`/`operator+` on device `__half` values resolve to real half-precision `__hmul`/`__hadd`, no promotion to `float` anywhere, and no WMMA/tensor-core instruction anywhere in the kernel.

Both raise the input magnitude from the earlier baseline's uniform range $(-256, 256)$ to $(-2000, 2000)$ — already sign-dithered, since a distribution symmetric about 0 draws each sign with equal probability. At this magnitude, individual products $a_i b_i$ can reach $\sim$4,000,000, about 61$\times$ past FP16's $\sim$65504 max finite value.

10.2 Results: host-side sanity check (independent of nixnan)

All four transpose combinations, 128^3, before any instrumentation is involved at all:

Kernel	finite	NaN	+Inf	−Inf
Tensor core, `float` accumulator (control)	16384/16384	0	0	0
Tensor core, `__half` accumulator	0/16384	0	≈8200	≈8180
SIMT, pure `__half` accumulate	0/16384	0	8114	8270

The `float`-accumulator control never overflows, on any of the four combinations, exactly matching Section 9's zero-exception runs. The `__half`-accumulator tensor-core kernel and the pure-half SIMT kernel both overflow *completely* — every one of the 16,384 output elements is ±Infinity, roughly split between signs, with zero NaN at this magnitude and seed. **This settles the question directly**: a tensor core can and does overflow FP16 once its accumulator is narrow; what protects `leimao-mm.cu`'s actual HMMA path is specifically its choice of a wide (`float`) accumulator, not anything inherent to tensor-core hardware. And ordinary SIMT-core FP16 arithmetic, with no tensor core involved at all, overflows the same way.

10.3 Results: caught by nixnan

Both files were rerun unsampled, default exception checking, and nixnan independently confirms each overflow at the exact instruction responsible:

Kernel	FP16 Infinity / −Infinity	Instruction
Tensor core, `__half` accumulator	16 sites / 7664 repeats (each sign)	`HMMA.16816.F16`
SIMT, pure `__half` accumulate	66 / 64 sites, 130,486 repeats (each sign)	`HFMA2`

The SIMT case is the cleanest possible confirmation of the underlying question: `HFMA2` (packed half-precision fused-multiply-add) is an ordinary CUDA-core instruction, not a tensor-core one — nixnan catches plain SIMT-core FP16 additions overflowing to Infinity, with zero tensor-core instructions anywhere in the kernel. The tensor-core case shows the matching FP32 epilogue arithmetic (the `alpha*acc+beta*c_prev` scale step, promoted to `float` for the multiply since `float` and `__half` have no unambiguous `operator*` together) inheriting and re-reporting the same Infinity coming out of the `__half` accumulator: 64 FP32 sites / 4032 repeats per sign, directly downstream of the 16 `HMMA.16816.F16` sites.

A buffer-reuse pitfall, caught and fixed during this experiment. An earlier version of `leimao-mm-128x128-overflow.cu` reused the same device output buffer across all four transpose combinations without clearing it between runs. Since the epilogue computes $\beta \cdot c_{\text{prev}}$ with β=0, this looks harmless — except $0 \times \text{NaN} = \text{NaN}$ in IEEE arithmetic, so once one combo's `__half`-accumulator run overflowed, its leftover NaN/Inf silently leaked into the *next* combo's "prior C" read, turning 3 of the 4 combinations' outputs into all-NaN rather than the true, combo-isolated all-Infinity result. Zeroing both output buffers with `cudaMemset` before every launch fixed this. Left in as a real, if minor, illustration of the same class of bug this whole tutorial is about: an exception from one computation silently contaminating a later, logically unrelated one through reused state.

11 Conclusion

For row-major A times row-major B (Section 1): FP16 works today, unmodified, via `leimao-mm.cu`'s existing `is_A_transpose=true, is_B_transpose=true` call. BF16 is fully supported by the same CUDA/hardware stack, confirmed directly in `mma.hpp`, and now has its own working file too, `leimao-tc-variant-bf16.cu` — a direct, same-shape port of the FP16 path with the element type

swapped and the accumulator forced to `float`. FP32 has no true full-precision tensor-core path at all; the tensor-core option is TF32 (an explicit, hardware-managed precision reduction on load, full-precision accumulate, smaller K-tile), and genuine full-precision FP32 GEMM needs an entirely separate, non-tensor-core SIMT kernel — which also now exists (`leimao-simt-variant-fp32.cu`, alongside its FP16 counterpart). Every GPU-side multiply-accumulate that exists in `leimao-mm.cu` itself runs on tensor cores; its only non-tensor-core matrix multiply is a CPU-side correctness check, not a GPU kernel.

Two real benchmarks put numbers on what this report otherwise only described structurally. Precision-matched (Section 6, FP16 both ways): tensor core ≈ 0.178 ms vs. SIMT ≈ 0.865 ms, $\approx 4.9\times$. Cross-precision (Section 7, the practical question of whether it's worth converting FP32 data to BF16 to use the tensor cores): FP32 SIMT ≈ 0.845 ms vs. BF16 tensor core ≈ 0.177 ms, $\approx 4.8\times$ — the same margin, and BF16's raw latency is indistinguishable from FP16's, exactly as Section 3's header-level finding (same tile shape, same accumulator type) predicts. Section 2's structural point (one HMMA/BMMA instruction does the work of 4096 scalar multiply-adds) and these measured $\approx$4.8–4.9$\times$ figures are two different kinds of evidence for the same conclusion, not the same number restated: the instruction count says why a gap should exist; the benchmarks say how large it actually is, for this kernel, this problem size, this GPU.